

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Data Mining (CO3029)

Topic

**Research Paper Classification
Using Textual and Metadata Features**

Lecturer: Vũ Ngọc Tú
Class: CC02
Students: Nguyễn Văn An - 2352013
Nguyễn Tấn Phát - 2352889

HO CHI MINH CITY, June 2026



Contents

1	Introduction	2
2	Dataset and Additional Data Crawling	3
2.1	Dataset Overview	3
2.2	Dataset Characteristics	3
2.3	Semantic Scholar Metadata Crawling	4
2.4	Retrieved Metadata Features	4
3	Exploratory Data Analysis and Preprocessing	6
3.1	Data Inspection	6
3.2	Missing Value Analysis	6
3.3	Label Distribution Analysis	6
3.4	Publication Year Analysis	8
3.5	Venue Analysis	8
3.6	Textual and Author Characteristics	12
3.7	Missing Value Handling	12
3.8	Text Preparation	12
3.9	Metadata Feature Preparation	12
4	Model Development Process	14
4.1	Establishing Baselines: Overfitting and Linear Models	14
4.2	Refining the Architecture: SciBERT, SPECTER2, and CORAL	14
4.3	The Local Compute Ceiling and LLM Zero-Shot Heuristics	14
4.4	Phase 2: Final Stacking Execution	14
5	Methodology of the Final Framework	15
5.1	Overall Framework	15
5.2	Metadata Enrichment & LLM Extraction	15
5.3	Phase 1: SPECTER2 with Adapters and AWP	15
5.3.1	CORAL (Consistent Rank Logits)	15
5.3.2	AWP (Adversarial Weight Perturbation)	16
5.4	Phase 2: Ridge Stacking Ensemble	17
5.5	Percentile Distribution Matching	17
6	Results and Discussion	18
6.1	Experimental Setup	18
6.2	Performance	18
6.3	Discussion and Analytical Insights	18
6.4	Lessons Learned	19
7	Conclusion and Future Work	20
8	Kaggle Screenshots	21



1 Introduction

Research paper classification is an important task in academic information management, enabling the organization, retrieval, and analysis of scientific literature. As the volume of published research continues to grow, automatic classification systems can assist researchers in discovering relevant work and identifying trends within large collections of documents.

This project was conducted as part of the Data Mining course assignment, based on the Kaggle Competition - Data Mining 252 Stage 2. The objective was to predict the ordinal category label of research papers in the domain of Answer Set Programming (ASP) using available metadata.

Rather than solely presenting the final classification architecture, this report comprehensively documents the entire iterative process that led to the final solution. It details the initial problem analysis, the rationale behind methodological pivots, and the step-by-step optimizations carried out to overcome the limitations of the competition dataset. The narrative highlights several key phases of the development journey:

- **Data Exploration and Enrichment:** Analyzing the constraints of the original dataset and detailing the rationale for engineering a custom metadata crawling pipeline via external academic APIs.
- **Establishing Baselines and Identifying Failures:** Documenting the initial experiments with traditional lexical models (TF-IDF) and analyzing why complex tree-based algorithms severely overfit the small dataset.
- **Iterative Architectural Improvements:** Explaining the transition toward domain-specific language models (SPECTER2), the adoption of ordinal regression (CORAL), and the necessity of regularization techniques like Adversarial Weight Perturbation (AWP).
- **Final Ensemble Synthesis:** Detailing how the reasoning capabilities of Large Language Models (LLMs) were integrated as zero-shot heuristics into a final, highly generalizable Ridge stacking ensemble.

The remainder of this report is organized to follow this chronological development process. Section 2 analyzes the dataset and the external metadata collection process. Section 3 presents the exploratory data analysis and preprocessing procedures. Section 4 discusses the iterative development process, highlighting the rationale behind each model pivot. Section 5 provides the technical blueprint of the final proposed framework. Finally, Section 6 reports the experimental results, and Section 7 concludes with lessons learned.



2 Dataset and Additional Data Crawling

2.1 Dataset Overview

The dataset used in this project was provided through the Kaggle Competition - Data Mining 252 Stage 2. The task focuses on the classification of research papers in the domain of Answer Set Programming (ASP) and related research areas. Each paper is represented by a set of metadata attributes and is associated with an ordinal category label ranging from 1 to 5.

The competition dataset is distributed across three files:

- **train.csv:** Training set containing paper metadata and corresponding target labels.
- **public_test.csv:** Test set used for public leaderboard evaluation.
- **private_test.csv:** Test set used for the final private leaderboard evaluation.

Each record contains the following attributes:

Table 1: Dataset attributes

Attribute	Description
id	Unique identifier of each paper
title	Title of the research paper
authors	List of authors
venue	Conference or journal venue
year	Publication year
doi	DOI or external reference identifier
label	Ordinal target label (1–5)

The objective of the competition is to predict the ordinal label of each research paper using the available metadata. Model performance is evaluated using the Quadratic Weighted Kappa (QWK) metric, which is designed for ordinal classification problems and assigns larger penalties to predictions that are further from the true label.

2.2 Dataset Characteristics

Although the dataset provides several useful metadata fields, the available information is relatively limited for accurately capturing the semantic content of a research paper. Important information such as abstracts, summaries, citation statistics, and reference information are not included in the original dataset.

Furthermore, many of the provided features are categorical or textual in nature, requiring additional preprocessing and feature engineering before they can be effectively utilized by machine learning models. For example, author names and publication venues must be transformed into numerical representations, while paper titles require text processing and semantic encoding.

These limitations motivated the use of external academic metadata sources to enrich the original dataset with additional information describing the content and impact of each paper.

2.3 Semantic Scholar Metadata Crawling

The original competition dataset provides only limited metadata, including paper title, authors, publication venue, publication year, and DOI information. To obtain richer semantic and bibliometric information, an additional metadata collection pipeline was developed using external academic databases.

The metadata retrieval process was performed for all papers in the training, public test, and private test sets. Each paper was first analyzed based on its DOI field to determine the appropriate retrieval source. Papers containing standard DOI identifiers were queried through the CrossRef API, while papers associated with Semantic Scholar URLs were retrieved directly from the Semantic Scholar Academic Graph API. This dual-source strategy maximized metadata coverage across the entire dataset.

To ensure reliable retrieval, the crawler implemented rate limiting, retry mechanisms, and error handling. CrossRef requests were limited to four requests per second, while Semantic Scholar requests were restricted to 0.8 requests per second to remain within the service limits. Exponential backoff was applied whenever rate-limit responses were encountered. Retrieved API responses were stored in JSONL format to preserve the complete raw metadata and facilitate reproducibility.

The Semantic Scholar API was configured to retrieve a broad set of academic metadata, including title, abstract, authors, venue, publication information, citation statistics, reference statistics, publication types, journal information, and automatically generated TLDR summaries. Similarly, CrossRef records were used to retrieve publication metadata and abstracts whenever available.

After collection, a metadata cleaning and extraction pipeline was applied. Raw abstracts were processed to remove XML tags, mathematical markup, LaTeX commands, and other formatting artifacts. Text fields were normalized and converted into a consistent format suitable for downstream processing. Structured metadata fields were then extracted and transformed into tabular features.

Finally, the cleaned metadata were converted into feature files and merged with the original competition dataset using the paper identifier. This process enriched the original dataset with additional textual and bibliometric information that was subsequently utilized during feature engineering and model development.

2.4 Retrieved Metadata Features

Several additional features were extracted from the Semantic Scholar metadata and incorporated into the modeling pipeline:



Table 2: Features extracted from external metadata sources

Feature	Description
abstract	Full paper abstract
tldr	Short paper summary
citationCount	Total citations received
influentialCitationCount	Influential citations received
referenceCount	Number of cited references
publisher	Publisher information
journal_name	Journal name when available
publication_date	Publication date
publication_type	Publication category

The abstract and TLDR fields were primarily used to enrich textual representations for transformer-based models and traditional text-based feature extraction methods. Citation-related attributes were incorporated as numerical metadata features after logarithmic transformation to reduce the influence of extreme values.

By combining the original competition metadata with externally collected Semantic Scholar information, the resulting dataset provides a richer representation of each paper, supporting both semantic understanding and metadata-driven learning approaches.

3 Exploratory Data Analysis and Preprocessing

3.1 Data Inspection

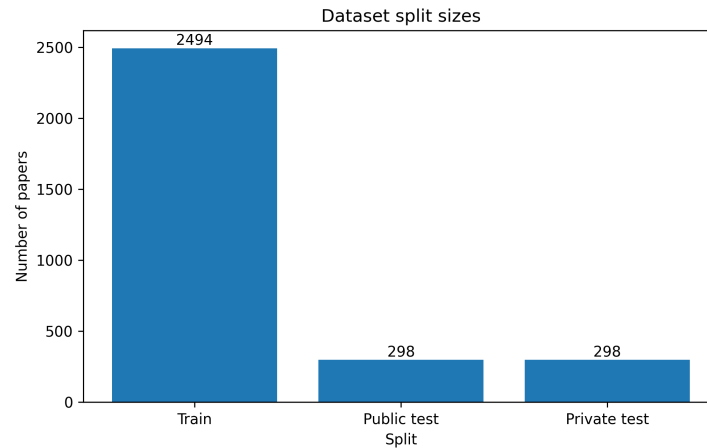


Figure 1: Dataset split sizes

The competition dataset consists of three data partitions: a training set containing 2,494 research papers, a public test set containing 298 papers, and a private test set containing 298 papers, as illustrated in **Figure 1**. The training set accounts for approximately 81% of the available data and serves as the primary source for model training and validation. Both test sets are considerably smaller and are reserved for leaderboard evaluation. The relatively limited training size motivates the use of careful cross-validation and feature engineering techniques to maximize generalization performance.

3.2 Missing Value Analysis

Missing value analysis revealed that the dataset is largely complete. As shown in **Figure 2**, the only attribute exhibiting a notable amount of missing data is the **authors** field. Approximately 7.7% of training samples, 6.38% of public test samples, and 7.38% of private test samples contain missing author information. In contrast, critical attributes such as title, venue, year, and DOI are fully available across all dataset splits.

Since author information is later utilized for feature engineering, missing author entries must be handled carefully during preprocessing. However, the overall completeness of the dataset suggests that missing values are unlikely to significantly affect model performance.

3.3 Label Distribution Analysis

The distribution of ordinal target labels in the training set is shown in **Figure 3**. The dataset exhibits a moderate class imbalance, with Label 1 representing the largest category (903 samples, 36.21%) and Label 5 representing the smallest category (272 samples, 10.91%). Labels 2, 3, and 4 account for 20.61%, 17.56%, and 14.72% of the dataset, respectively.

Although all five classes are reasonably represented, the imbalance indicates that naive accuracy may not adequately reflect model performance. Furthermore, the ordinal nature of the labels

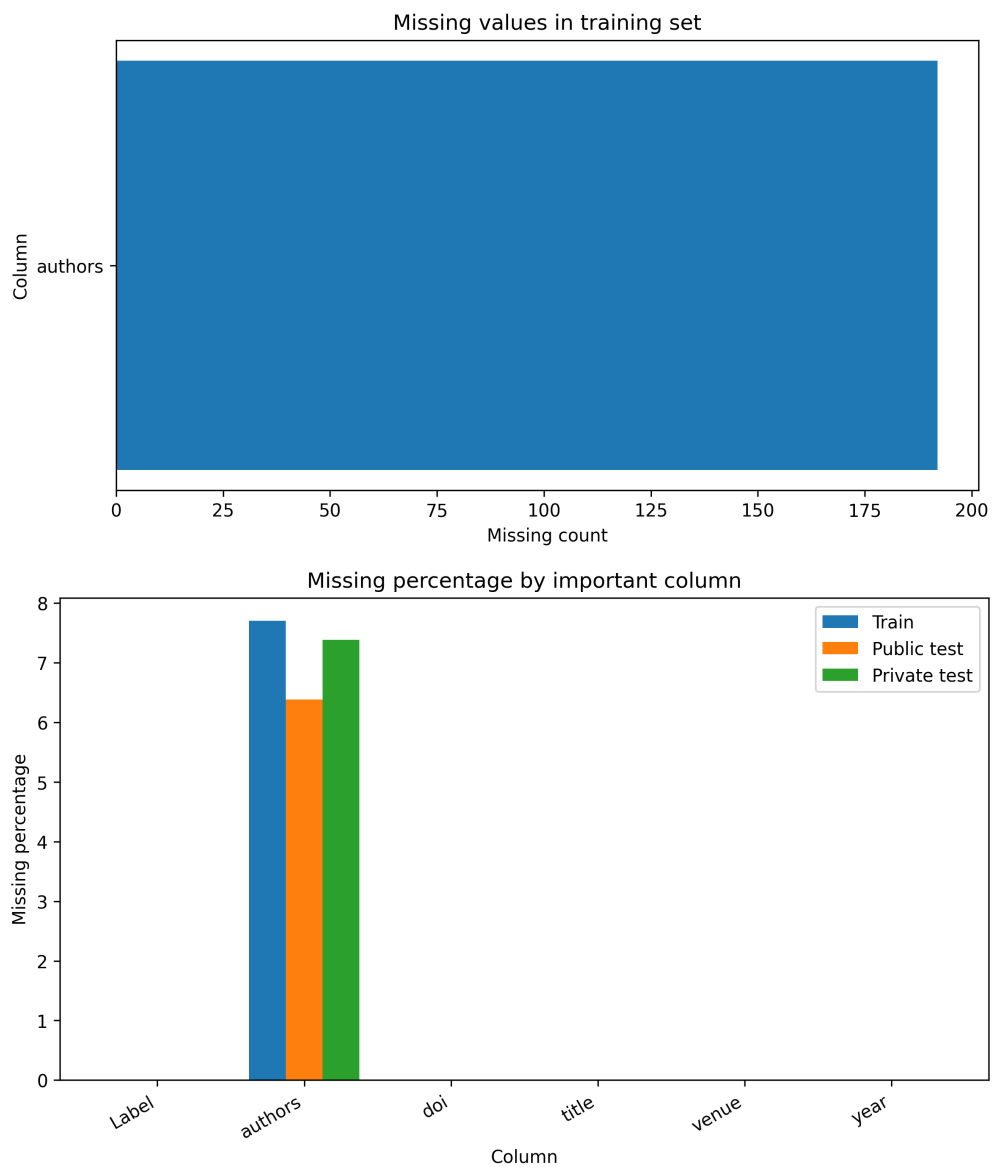


Figure 2: Missing value analysis

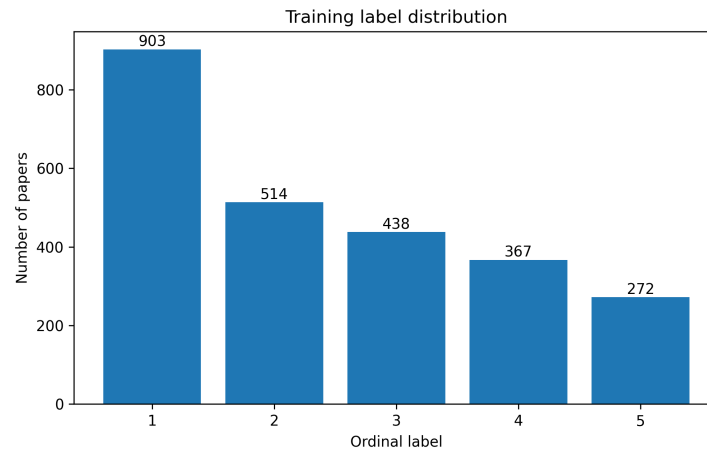


Figure 3: Label distribution

suggests that prediction errors should not be treated equally. Consequently, the competition evaluates submissions using the Quadratic Weighted Kappa (QWK) metric and the training process employs stratified cross-validation to preserve label proportions across folds.

3.4 Publication Year Analysis

Figure 4 illustrates the distribution of publication years across all dataset splits. Most papers were published between 2016 and 2025, with publication counts generally increasing in more recent years. The training set contains the largest concentration of papers in 2024 and 2025, suggesting that the dataset primarily reflects contemporary research trends within the ASP community.

The relationship between publication year and target labels is further examined in **Figure 4**. While Label 1 remains the dominant category across all years, the relative proportions of other labels vary over time. This observation suggests that publication year may provide useful contextual information and could serve as a valuable metadata feature for classification.

3.5 Venue Analysis

Venue information is one of the most informative metadata attributes in the dataset. **Figure 5** shows that papers originate from only five major venues: CAV, KR, LICS, ICLP, and LPNMR. Among these, CAV contains the largest number of papers (878), followed by KR (812) and LICS (791).

The label composition of each venue, shown in the right side, reveals substantial differences in class distributions. For example, LICS is heavily dominated by Label 1 papers, whereas KR exhibits a more balanced distribution across all labels. Such variations indicate that venue information is strongly correlated with research topics and publication characteristics. Therefore, venue-based features are expected to contribute significantly to classification performance.

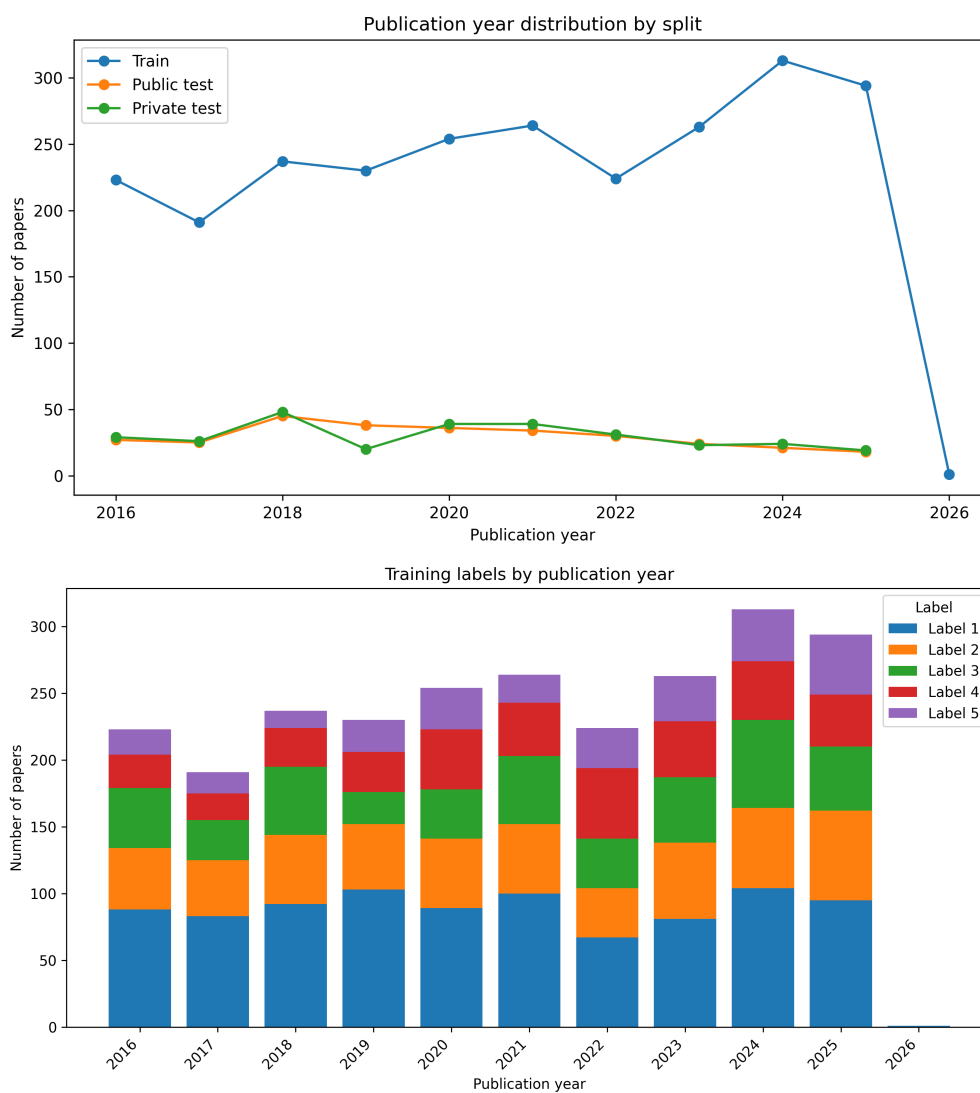


Figure 4: Publication year distribution

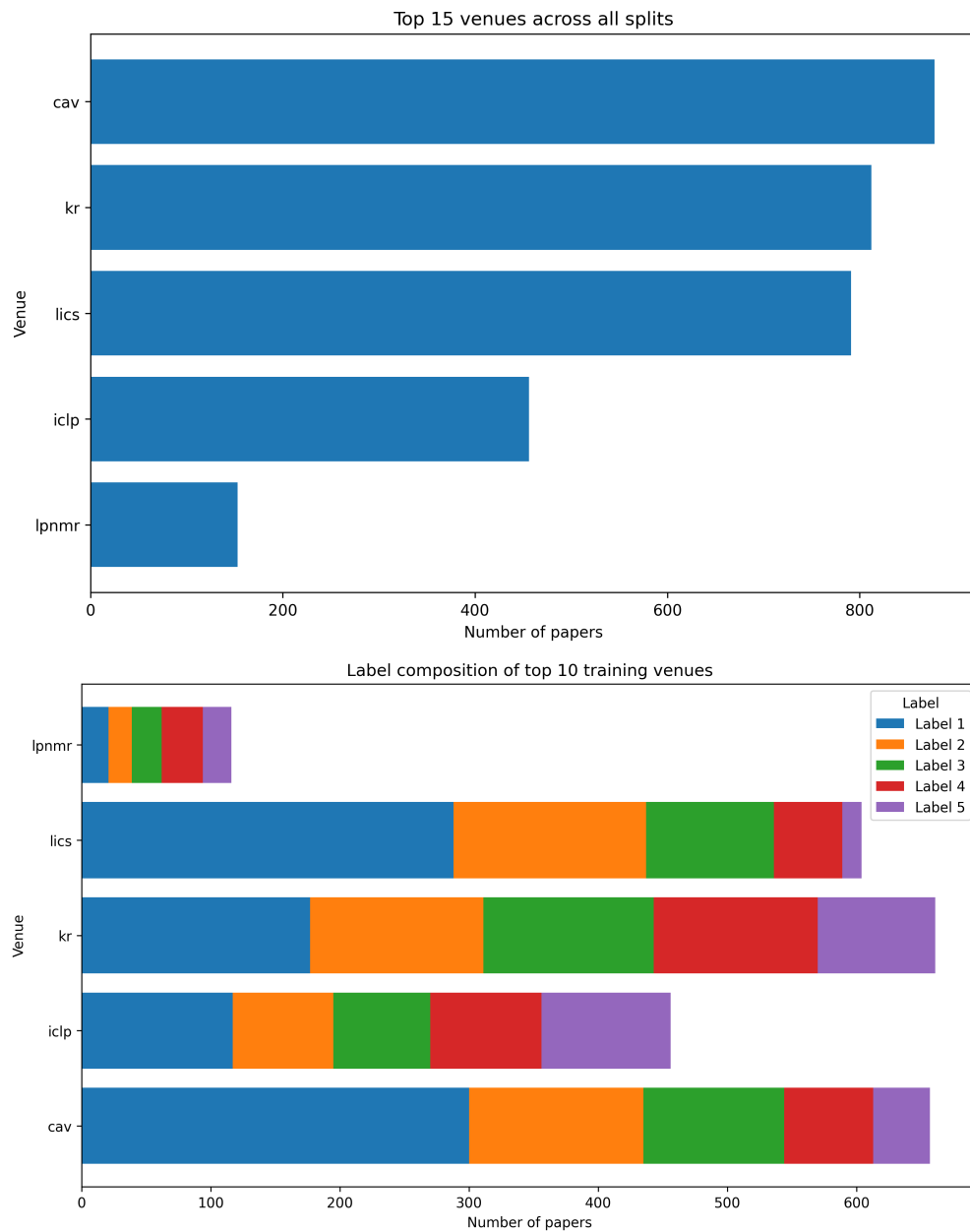


Figure 5: Venue distribution

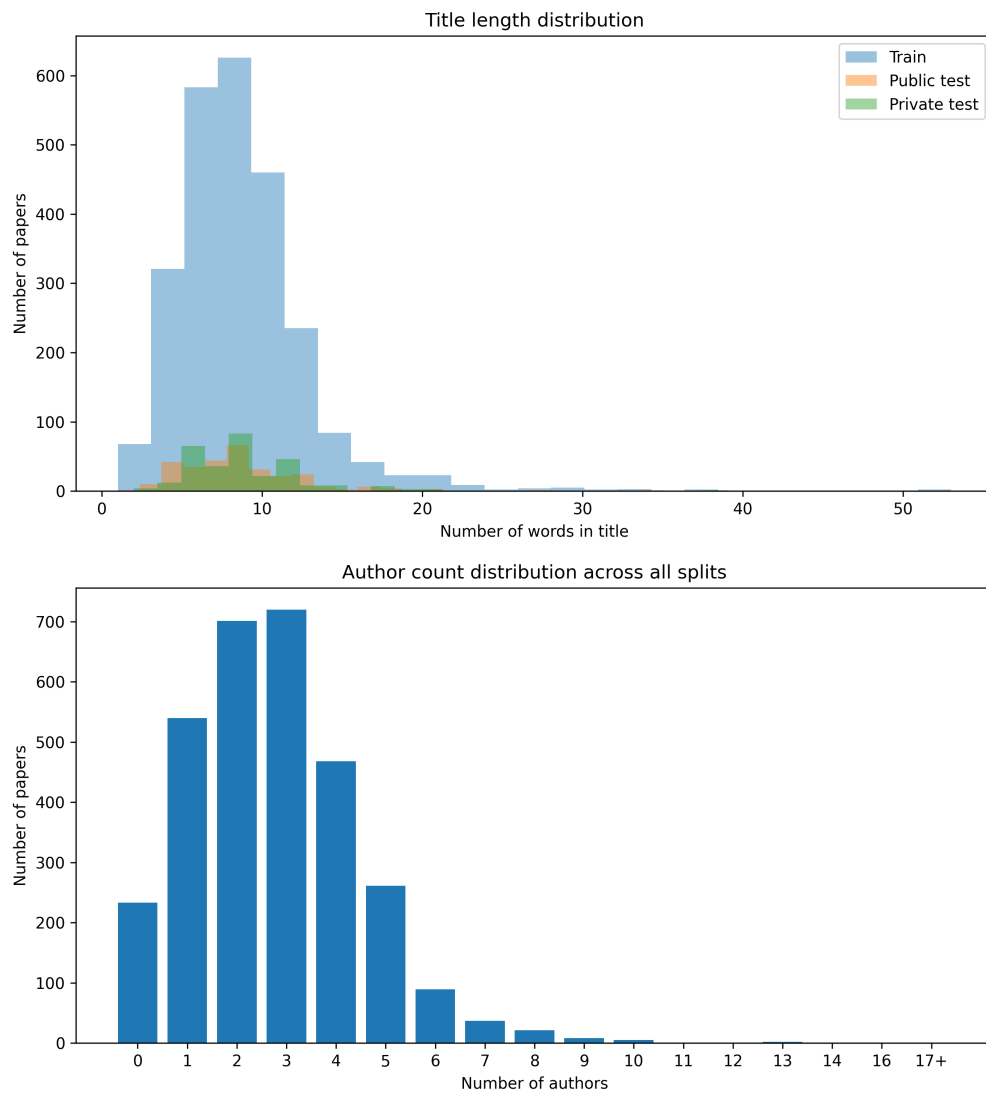


Figure 6: Title length and Author count

3.6 Textual and Author Characteristics

The textual characteristics of paper titles were analyzed by measuring title length in terms of word count. As shown in **Figure 6**, most titles contain between 5 and 12 words, with the distribution peaking around 7–10 words. A small number of titles exceed 20 words, indicating the presence of a few unusually long paper titles.

Author statistics are presented in **Figure 6**. Most papers are written by two to four authors, with three-author papers being the most common. Single-author papers are less frequent, while papers with more than six authors are relatively rare. The presence of recurring authors and varying collaboration sizes suggests that author-related information may provide additional predictive signals. Consequently, author-based feature engineering techniques are explored in the modeling phase.

3.7 Missing Value Handling

Based on the exploratory analysis presented in **Section 3.2**, the dataset contains very few missing values. The only attribute with a notable amount of missing data is the **authors** field, with approximately 7% of records missing author information across all dataset splits. In contrast, critical attributes such as title, venue, year, and DOI are fully populated.

To ensure consistency during feature engineering, missing author entries were replaced with empty strings. This approach preserves all records while allowing author-based feature extraction methods to be applied uniformly across the dataset. Since the proportion of missing values is relatively small, no records were removed during preprocessing.

3.8 Text Preparation

The original dataset provides only paper titles as textual information. To enrich the semantic content available to the models, additional metadata collected from Semantic Scholar, including abstracts and TLDR summaries, were incorporated into the preprocessing pipeline.

Prior to feature extraction, textual fields were standardized by removing excessive whitespace and replacing missing values with empty strings. Multiple textual sources were then combined into a unified document representation consisting of:

- Publication venue
- Paper title
- TLDR summary
- Abstract

The resulting text representation provides a richer description of each paper than the original title alone, allowing both transformer-based models and traditional text mining techniques to capture more comprehensive semantic information.

3.9 Metadata Feature Preparation

In addition to textual information, several metadata attributes were prepared for downstream modeling to capture the broader academic context of each paper. Because the original competition dataset provided limited information for accurately capturing a paper’s semantic content and impact, external academic metadata was integrated.



Numerical features such as citation count, influential citation count, and reference count were incorporated to serve as bibliometric indicators of a paper's content and influence. Since these citation-related variables exhibit highly skewed distributions, logarithmic transformations were applied to reduce the influence of extreme values and improve numerical stability.

Author information was further processed to derive additional features, including the number of authors associated with each paper. Publication venue information was preserved as a categorical attribute, while DOI availability was converted into a binary indicator feature.

Finally, all metadata features were organized into a structured representation that could be integrated with textual embeddings and traditional machine learning features in the subsequent modeling stage. However, because adding metadata can sometimes introduce noise and lower the overall score, we handled this curated feature set cautiously. During the evaluation phase, we simply tested the models both with and without the metadata module to ensure we only kept it if it actually improved the classification performance.

4 Model Development Process

4.1 Establishing Baselines: Overfitting and Linear Models

Our initial approach established a baseline using a Term Frequency-Inverse Document Frequency (TF-IDF) vectorizer paired with an XGBoost classifier, yielding a public QWK score of 0.51364. Attempting to improve this by concatenating native metadata caused the score to plummet to 0.40666. This severe degradation indicated that complex tree-based decision models like XGBoost were drastically overfitting our relatively small dataset. Swapping XGBoost for a regularized linear model (TF-IDF + Ridge) successfully brought our public score up to 0.59694, proving that simpler architectures generalize better on this dataset.

4.2 Refining the Architecture: SciBERT, SPECTER2, and CORAL

Shallow lexical models treat tokens independently, failing to capture latent semantics or domain authority. This motivated a pivot to models pre-trained on internet-scale academic data. We first transitioned to SciBERT, which provided a massive performance improvement over the linear baselines. However, to push performance even further, we transitioned our final architecture to SPECTER2, as it is specifically designed for scientific document processing.

To optimize training efficiency and stability on this final transformer, we utilized the `adapters` library to inject a parameter-efficient classification head rather than unfreezing the entire base model. Simultaneously, we pivoted to an ordinal regression approach using CORAL (Consistent Rank Logits) loss. Standard classification penalizes predicting a 1 when the answer is 2 exactly the same as predicting a 1 when the answer is 5. CORAL mathematically aligned the model with the ordinal nature of the labels. To further combat overfitting, we introduced Adversarial Weight Perturbation (AWP) into the training loop, forcing the model to learn generalized representations without memorizing the small dataset.

4.3 The Local Compute Ceiling and LLM Zero-Shot Heuristics

After scaling to 10 folds, we hit a hard performance ceiling. We realized base transformer models lacked the reasoning depth to evaluate qualitative metrics like a human reviewer. However, lacking the computational power to fine-tune large language models (LLMs), our only viable option was inference. We used Qwen2.5-7B-Instruct as a zero-shot evaluator, passing the text through a strict prompt to output a JSON evaluating Novelty, Clarity, Rigor, Depth, and Impact on a 1-10 scale.

4.4 Phase 2: Final Stacking Execution

To integrate these insights, we implemented a dedicated two-phase stacker. We first isolated the continuous out-of-fold predictions from our robust SPECTER2 adapter model. We then concatenated these logits directly with the five LLM-derived heuristic scores and trained a final Ridge meta-learner on this unified matrix. Applying percentile-based thresholding to match the dataset's natural distribution produced our most robust and generalizable model, securing our highest private leaderboard score.

5 Methodology of the Final Framework

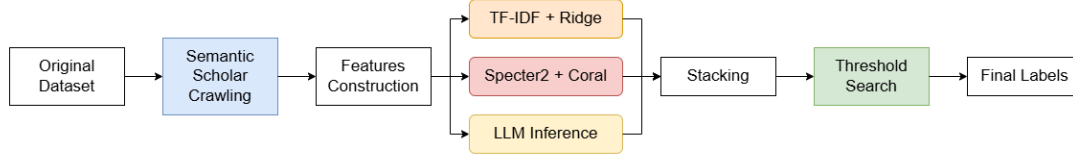


Figure 7: Pipeline of our final architecture

5.1 Overall Framework

The proposed research paper classification framework operates in two distinct phases. Phase 1 involves extracting Out-of-Fold (OOF) continuous logits using a transformer-based ordinal regression model enriched with structural metadata. Phase 2 concatenates these logits with structured heuristic scores generated by a Large Language Model (LLM) and processes them through a Ridge regression meta-learner. Final discrete labels are assigned using cumulative distribution matching.

5.2 Metadata Enrichment & LLM Extraction

The original dataset contains limited information. To construct a semantic representation, metadata was retrieved using the Semantic Scholar API, expanding the feature set to include abstracts, TLDR summaries, and bibliometric statistics (citation count, influential citations, reference count).

To capture high-level qualitative attributes, Qwen2.5-7B-Instruct was utilized as a zero-shot evaluator. The LLM processed the concatenated venue, title, and abstract of each paper to output a structured JSON assessing five metrics on a 1-10 scale: Novelty, Clarity, Rigor, Depth, and Impact.

5.3 Phase 1: SPECTER2 with Adapters and AWP

The primary base learner utilizes `allenai/specter2_base`, specifically designed for scientific document processing. Rather than updating the entire model, parameter-efficient fine-tuning was achieved by loading the `specter2_classification` adapter head.

5.3.1 CORAL (Consistent Rank Logits)

The classification task was formulated as ordinal regression utilizing the Consistent Rank Logits (CORAL) framework. Standard cross-entropy loss assumes that classes are independent, which fails to penalize a misclassification between adjacent ranks differently than a misclassification between distant ranks. Because the competition target is an ordered scale from 1 to 5, we decomposed the K -class problem into $K - 1$ independent binary classification tasks.

For a given paper with a true ordinal label $y \in \{1, 2, 3, 4, 5\}$, we construct a binary target vector

where the k -th element r_k indicates whether the label exceeds rank k :

$$r_k = \begin{cases} 1 & \text{if } y > k \\ 0 & \text{if } y \leq k \end{cases} \quad \text{for } k \in \{1, 2, 3, 4\} \quad (1)$$

To ensure rank-monotonicity and consistent confidence scores, CORAL utilizes a shared-weight architecture. The transformer outputs a single continuous feature logit, $f(x)$, for the input paper. The probability of the paper's rating exceeding rank k is calculated using the sigmoid function σ and an independent, strictly ordered bias term b_k for each task:

$$\hat{P}(y > k) = \sigma(f(x) + b_k) \quad (2)$$

The model is trained simultaneously on these 4 tasks using Binary Cross Entropy with Logits Loss (BCEWithLogitsLoss). The total loss $\mathcal{L}_{\text{CORAL}}$ is the sum of the binary cross-entropies:

$$\mathcal{L}_{\text{CORAL}} = - \sum_{k=1}^{K-1} \left[r_k \log(\hat{P}(y > k)) + (1 - r_k) \log(1 - \hat{P}(y > k)) \right] \quad (3)$$

During inference, the final continuous prediction is obtained by summing the predicted probabilities across all $K - 1$ thresholds and adding 1:

$$\hat{y}_{\text{continuous}} = 1 + \sum_{k=1}^{K-1} \hat{P}(y > k) \quad (4)$$

5.3.2 AWP (Adversarial Weight Perturbation)

To combat overfitting within the 10-fold cross-validation scheme, Adversarial Weight Perturbation (AWP) was injected into the training loop beginning at epoch 2. Deep neural networks fine-tuned on small datasets are notoriously prone to finding sharp local minima in the weight loss landscape, leading to poor robust generalization. AWP explicitly regularizes the flatness of this weight loss landscape by solving a min-max optimization problem that forces the optimizer into a flatter, more robust minimum.

Standard training seeks to minimize the loss $\mathcal{L}$ with respect to the model weights w . In contrast, AWP assumes an adversary is actively perturbing the weights by a small continuous amount v , constrained by a perturbation budget γ , to maximize the training error. The AWP objective minimizes this worst-case perturbed loss:

$$\min_w \max_{\|v\| \leq \gamma} \mathcal{L}(w + v) \quad (5)$$

In practice, this is implemented during the backward pass in two steps. First, the inner maximization is approximated by calculating the gradient of the loss with respect to the weights, pushing the perturbation v in the direction of steepest ascent:

$$v = \gamma \frac{\nabla_w \mathcal{L}(w)}{\|\nabla_w \mathcal{L}(w)\|} \quad (6)$$

Second, the outer minimization updates the actual model weights using gradient descent on the adversarial loss $\mathcal{L}(w + v)$. By explicitly penalizing gradient variations, AWP guarantees that the model learns generalized semantic representations rather than memorizing hyper-specific lexical noise in the training set.

5.4 Phase 2: Ridge Stacking Ensemble

The stacking ensemble linearly combines the semantic patterns captured by Phase 1 with the structured heuristics of the LLM. The Phase 2 training matrix was constructed by concatenating the continuous OOF predictions from the SPECTER2 model with the 5 LLM-derived evaluation scores.

A Ridge Regression model ($\alpha = 10.0$) served as the meta-learner. Ridge regression was specifically selected over tree-based ensembles to provide strict L2 regularization, preventing the model from memorizing the LLM heuristics while effectively balancing the feature weights.

5.5 Percentile Distribution Matching

Because the Phase 2 Ridge meta-learner outputs continuous scalar values, decision thresholds are required to map these scores back to the discrete ordinal labels (1-5). Instead of utilizing uniformly spaced boundaries, which often compress minority classes and introduce bias in imbalanced datasets, a Cumulative Distribution Function (CDF) matching strategy was employed.

This approach structurally guarantees that the final distribution of predicted labels mathematically mirrors the class imbalance of the ground-truth data. Let N be the total number of training samples and N_i be the number of samples belonging to class $i \in \{1, 2, 3, 4, 5\}$. The empirical probability of each class is $P(Y = i) = \frac{N_i}{N}$. The Cumulative Distribution Function $F(k)$, representing the fraction of the training data belonging to class k or lower, is defined as:

$$F(k) = P(Y \leq k) = \sum_{i=1}^k P(Y = i) \quad (7)$$

Let $\hat{y}$ represent the array of continuous out-of-fold predictions generated by the meta-learner. To map these continuous scores to discrete labels, we require four decision boundaries, τ_k , for $k \in \{1, 2, 3, 4\}$. Rather than optimizing these boundaries through arbitrary spacing or exhaustive grid search, we locate the exact value in $\hat{y}$ that splits the data at the cumulative probability $F(k)$. Thus, the threshold τ_k is strictly defined as the $F(k)$ -th percentile of the continuous validation predictions:

$$\tau_k = \text{Percentile}(\hat{y}, 100 \times F(k)) \quad (8)$$

During inference, a new continuous prediction s is assigned its final discrete ordinal label $\hat{y}_{\text{final}}$ by evaluating it against these mathematically aligned boundaries:

$$\hat{y}_{\text{final}} = 1 + \sum_{k=1}^4 \mathbb{I}(s > \tau_k) \quad (9)$$

where $\mathbb{I}$ is the indicator function evaluating to 1 if the condition is true and 0 otherwise. By strictly enforcing this percentile alignment, the model conservatively respects the known ground-truth imbalances, thereby mathematically minimizing the disproportionate penalty errors inherent to the Quadratic Weighted Kappa (QWK) metric.

6 Results and Discussion

6.1 Experimental Setup

Experiments were conducted using 10-fold Stratified Cross Validation to ensure consistent label distributions across folds. The Quadratic Weighted Kappa (QWK) metric was used as the primary evaluation criterion. Model selection and threshold optimization were performed using out-of-fold predictions generated during cross-validation.

6.2 Performance

Table 3: Cross-validation performance of implemented strategies

Model	Private Score	Public Score
TF-IDF + XGBoost	0.49218	0.51364
+ Metadata	0.50484	0.40666
TF-IDF + Ridge	0.59880	0.59694
SciBERT CV (5 folds)	0.68706	0.66273
SPECTER2 CV (5 folds) + CORAL	0.68350	0.66850
+ TF-IDF + Ridge stacking	0.68892	0.66927
+ Metadata	0.67064	0.68118
SPECTER2 CV (10 folds) + CORAL	<i>skipped</i>	<i>skipped</i>
+ TF-IDF + Ridge stacking	<i>skipped</i>	<i>skipped</i>
+ Metadata	0.67901	0.69377
+ Qwen Score	0.69131	0.69265

Table 3 presents the cross-validation performance of the individual models evaluated in this study. The final two-phase stacking ensemble, which integrated SPECTER2 adapter predictions with LLM-generated heuristics, achieved the strongest overall performance.

6.3 Discussion and Analytical Insights

The experimental results highlight several critical insights regarding the behavior of machine learning models on small, specialized academic datasets. Rather than immediately achieving high performance, the final architecture was the result of systematically diagnosing and overcoming specific model failures.

The Overfitting Paradigm of Tree-Based Models

Our initial experiments utilized an XGBoost classifier paired with TF-IDF features. While XGBoost is traditionally a powerful baseline, it failed catastrophically when metadata features were concatenated, dropping the public score to 0.40666. This provided a crucial analytical insight: with a training size of fewer than 2,500 samples, the decision trees were memorizing noise and hyper-specific lexical patterns rather than learning generalizable qualities. This necessitated the pivot to Ridge regression. By enforcing strict L2 regularization, the Ridge model penalized large feature weights, proving that in small-data regimes, restricting model complexity is mathematically more effective than adding feature depth.

Optimizing Transformer Performance

Transitioning to domain-specific transformers yielded our largest performance improvement, beginning first with SciBERT before finalizing the architecture with SPECTER2. While these base models generalized quite well on their own, relying purely on standard fine-tuning was not enough to secure a top position on the highly competitive Kaggle leaderboard. To squeeze out that critical extra performance without overfitting the small 2,500-sample dataset, we integrated Adversarial Weight Perturbation (AWP) into the training loop. By dynamically calculating the gradient direction that maximizes the loss and perturbing the weights in that direction, AWP forced the optimizer to find a flatter, more robust loss landscape. It allowed us to extract points of leaderboard performance that standard training could not achieve.

Aligning Loss Functions with Ordinality

A major turning point in our development process was the realization that standard cross-entropy loss fundamentally misaligned with the Quadratic Weighted Kappa (QWK) evaluation metric. Standard classification treats a misclassification between a 1 and a 2 identically to a misclassification between a 1 and a 5. To solve this, we formulated the architecture using Consistent Rank Logits (CORAL). By decomposing the 5-class target into 4 independent binary classifiers, we mathematically forced the model to recognize the ordered nature of the labels. This directly mitigated severe penalty errors and stabilized the model's predictive distribution.

Strategic Feature Isolation via Stacking

Our final breakthrough involved integrating the LLM-derived heuristic scores. Initially, combining dense semantic embeddings with structured heuristics in a single layer risked dimensional dominance, where the larger feature space overshadows the smaller one. Our analysis led to a two-phase stacking strategy. By isolating the deep representation learning in Phase 1 (SPECTER2 logits) and combining it linearly with the LLM scores in Phase 2 using a Ridge meta-learner, we allowed each feature type to contribute proportionally.

Threshold Optimization via CDF Matching

Finally, optimizing the continuous outputs of the Phase 2 meta-learner into discrete ordinal labels presented a mathematical challenge. Uniform thresholds often compress minority classes. Our insight was to utilize Cumulative Distribution Function (CDF) matching. By strictly aligning our prediction thresholds with the exact percentiles of the training set's class distribution, we guaranteed that our model conservatively respected the known ground-truth imbalances.

6.4 Lessons Learned

Several key structural lessons were identified during model development:

- **Necessity of Domain-Specific Pre-training:** Generic lexical models fail to capture deep academic contexts. Models explicitly pretrained on scientific corpora, combined with parameter-efficient fine-tuning via adapters, are essential for domain understanding.
- **Efficacy of Ordinal Modeling and CDF Matching:** Treating the target as a continuous ordinal variable rather than distinct categories avoids disproportionate penalization. Aligning continuous predictions to the training set's Cumulative Distribution Function (CDF) guarantees structural parity and drastically improves the final metric.
- **LLMs as Feature Extractors:** While local computational limits restrict the direct fine-tuning of large language models, employing an LLM as a zero-shot evaluator for specific qualitative metrics (e.g., Rigor, Novelty) provides robust, heuristic features that effectively ground the final ensemble.



7 Conclusion and Future Work

This project presents a robust machine learning framework for ordinal research paper classification, effectively combining semantic representations from domain-specific language models (SPECTER2) with large language model (LLM) heuristics. By leveraging parameter-efficient adapters and adversarial training, the proposed two-phase stacking ensemble overcomes the limitations of small dataset sizes to deliver highly generalizable predictions. Ultimately, integrating heterogeneous feature sources proved critical for capturing both the linguistic nuances and the academic context necessary for accurate quality assessment.

Despite these strong results, several limitations present opportunities for future work. First, local computational constraints restricted the use of the LLM to zero-shot inference. Future research could explore parameter-efficient fine-tuning (such as LoRA) of large language models directly on the ordinal evaluation task. Second, the current text features rely primarily on abstract-level information and aggregated citation statistics. Moving forward, incorporating full-text PDF parsing would allow the models to conduct a more granular assessment of methodological rigor and experimental depth. Finally, research papers do not exist in isolation. Representing the dataset as a comprehensive citation network and applying Graph Neural Networks (GNNs) could capture the latent relational context between authors and publications, further enhancing the framework's ability to evaluate long-term academic impact.



8 Kaggle Screenshots

General

TEAM NAME

AN NGUYỄN VĂN

This name will appear on your team's leaderboard position.

Figure 8: Team name

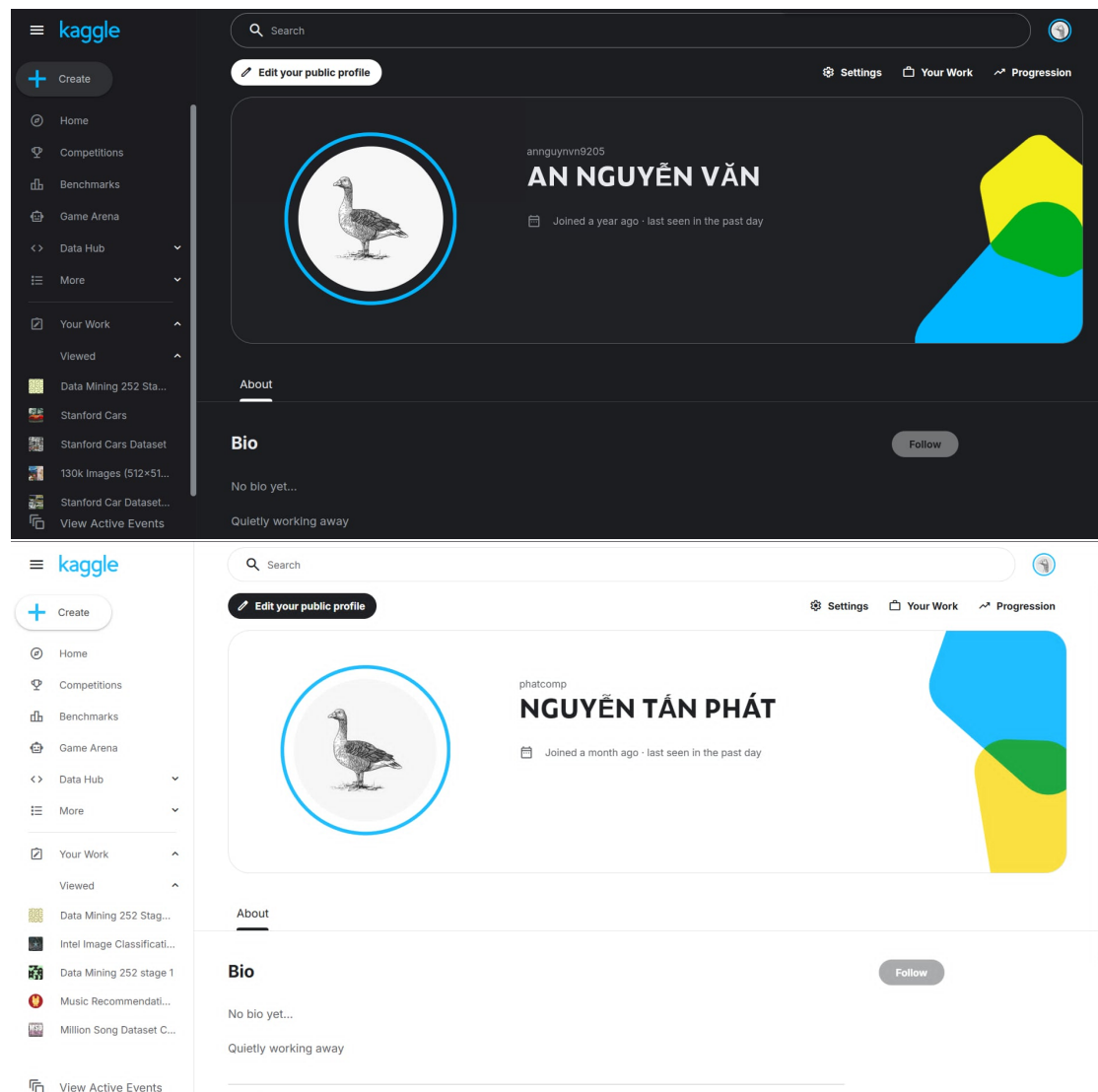


Figure 9: Team members

Overview	Data	Discussion	Leaderboard	Rules	Team	Submissions
20	▼ 2	QUÂN NGUYỄN MINH		0.69647	4	14d
21	▼ 16	K-H-N		0.69518	27	18d
22	▼ 2	AN NGUYỄN VĂN		0.69131	47	6d
23	▼ 2	NGUYỄN TẤN KHOA		0.69013	88	6d

Figure 10: Team's private leaderboard ranking (rank 22)